

Programowanie Obiektowe

Wprowadzenie

Dominik Kasprowicz

Instytut Mikroelektroniki i Optoelektroniki
Politechniki Warszawskiej

5 marca 2026

Spis treści

- 1 Java – maszyna wirtualna
- 2 Java – styl, instrukcje sterujące, operatory

Spis treści

- 1 Java – maszyna wirtualna
- 2 Java – styl, instrukcje sterujące, operatory

- Lata 1991–1995 – pierwsze prace nad językiem (Sun Microsystems).
- „Napisz raz, uruchamiaj gdziekolwiek”.
 - Właściwie: „*Skompiluj* raz, uruchamiaj gdziekolwiek”.
- Docelowa platforma – dekodery i piloty TV oraz podobne urządzenia.
 - Ówczesny sprzęt nie sprostał wyzwaniu, ale...
 - Powstało wiele aplikacji desktopowych i appletów osadzonych na stronach www.

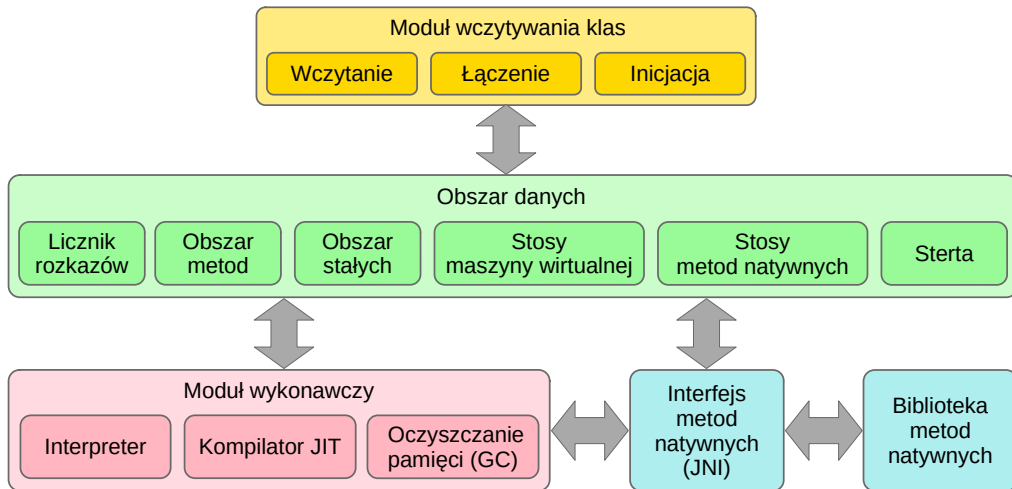
Java – założenia

- ❶ Język obiektowy.
- ❷ Możliwość uruchamiania raz skompilowanego programu na różnych platformach sprzętowych i różnych systemach operacyjnych.
- ❸ Wsparcie dla komunikacji sieciowej.
- ❹ Możliwość bezpiecznego uruchamiania kodu pobranego ze zdalnych źródeł.
- ❺ Wykorzystanie „dobrych” elementów istniejących języków obiektowych.

Maszyna wirtualna Javy – JVM

- **Maszyna wirtualna Javy (JVM)** to program działający na określonej platformie sprzętowej, interpretujący **kod bajtowy** (ang. *bytecode*) – ok. 200 rozkazów.
- JVM wykona każdy poprawny kod bajtowy...
 - ...będący efektem kompilacji kodu Java, lecz także języków Scala, Groovy, Kotlin...
 - Wiele implementacji JVM (i kompilatorów):
 - OpenJDK (open source, najpopularniejsza),
 - Java SE Development Kit (Oracle), **GraalVM** (Oracle), Corretto (Amazon) i wiele innych.
- JVM może także korzystać z metod **natywnych**: efektów kompilacji kodu np. języka C do kodu maszynowego *konkretnej maszyny* (nieprzenośnych).

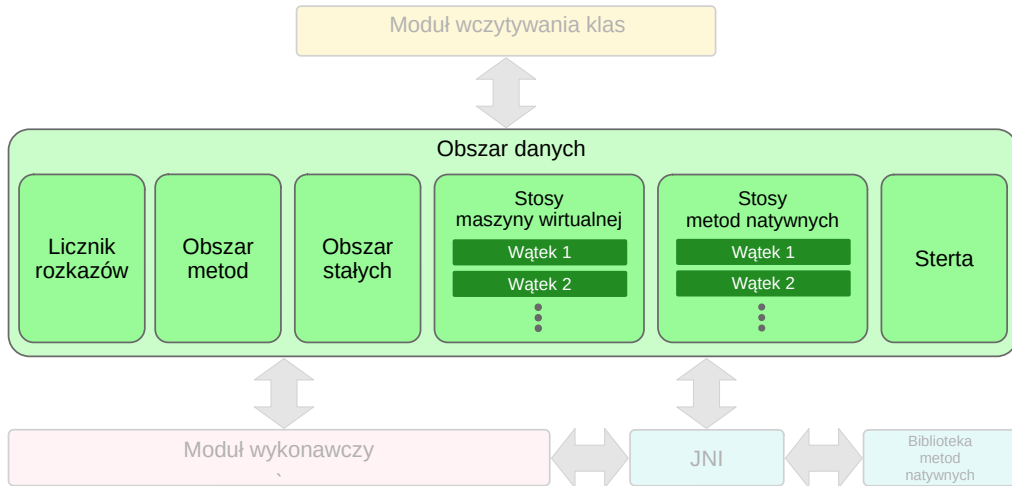
JVM – struktura



Moduł wczytywania klas – *Class Loader*

- **Wczytanie** (ang. *loading*) kodu bajtowego klasy do pamięci i utworzenie na jego podstawie obiektu klasy **Class**.
- **Łączenie** (ang. *linking*):
 - **Weryfikacja** definicji klasy, tzn. sprawdzenie:
 - poprawności strukturalnej kodu bajtowego,
 - zgodności kodu bajtowego z wersją maszyny wirtualnej (kod przeznaczony dla nowej maszyny nie zostanie uruchomiony na jej starszej wersji),
 - obecności w pamięci wszystkich innych potrzebnych klas,
 - prób wykonania nieprawidłowych operacji: używania niezainicjowanych zmiennych, przypisań do zmiennych niepoprawnych typów, zmian stałych, dziedziczenia po finalnych klasach, przeciążania finalnych metod itp.
 - **Przygotowanie**: tworzenie pól statycznych i ich inicjacja wartościami domyślnymi.
 - **Rozwiązanie** (ang. *resolution*): zastąpienie symbolicznych referencji adresami klas i stałych.
- **Inicjacja** – wywołanie konstruktorów klas, inicjacja zmiennych statycznych.

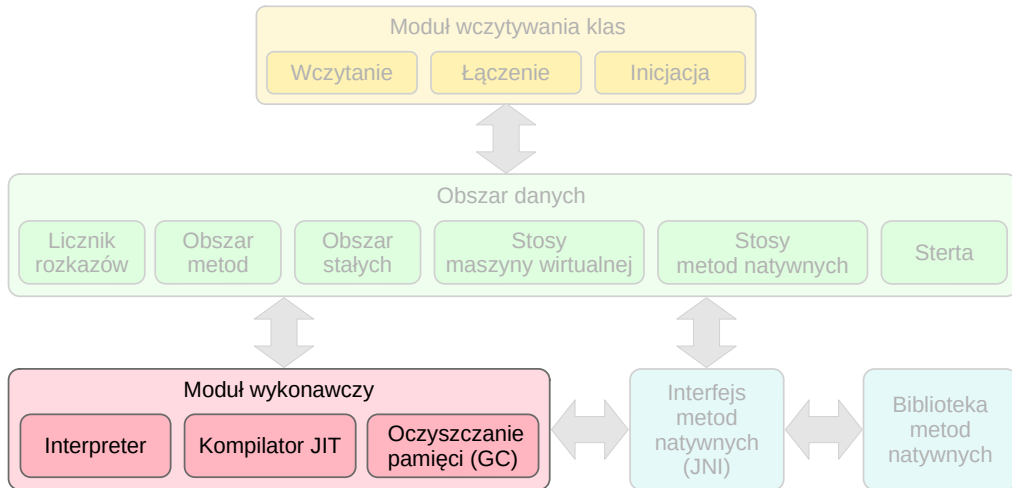
JVM – obszar danych (ang. *Runtime Data Areas*)



JVM – obszar danych (ang. *Runtime Data Areas*)

- **Licznik rozkazów** (ang. *PC Register*) – osobny dla każdego wątku; tylko dla instrukcji kodu bajtowego (nie natywnych).
- **Obszar metod** (ang. *Method Area*) – kod metod, informacja o typach i nazwach pól.
- **Obszar stałych** (ang. *Run-Time Constant Pool*) – wartości stałych.
- **Stosy maszyny wirtualnej** (ang. *JVM Stacks*) – przechowują:
 - ramki wywołań funkcji z kodu bajtowego,
 - lokalne zmienne typów podstawowych.
- **Stosy metod natywnych** (ang. *Native Method Stacks*) – używane, gdy jeśli JVM korzysta z metod **natywnych**: skompilowanych do kodu maszynowego.
- **Sterna** (ang. *Heap*) – wspólna dla wszystkich wątków
 - przechowuje typy referencyjne: obiekty i tablice.

JVM – moduł wykonawczy (ang. *Execution Engine*)



JVM – moduł wykonawczy (ang. *Execution Engine*)

- **Interpreter**

- Odczytuje kolejne wiersze kodu bajtowego, tłumaczy na kod natywny i wykonuje.
- Brak prób optymalizacji, np. zapamiętania kod natywnego dla wykonanego już raz kodu bajtowego.

- **Kompilator Just-in-Time (JIT)**

- Analizuje kod bajtowy podczas jego wykonania.
- Identyfikuje fragmenty kodu najsilniej wpływające na wydajność („*hot spots*”).
- Optymalizuje kod.

- **Moduł czyszczenia pamięci** (ang. *Garbage Collector, GC*)

- Usuwa z pamięci nieużywane obiekty.
- Jest *jedynym* mechanizmem usuwania obiektów i odzyskiwania pamięci.
- W danej JVM z reguły dostępnych kilka różnych strategii GC.

Kompilator Just-in-Time (JIT)

- Na bieżąco **profiluje** kod bajtowy – zbiera statystyki wydajności podczas wykonywania kodu bajtowego.
- Podejmuje próby optymalizacji „często” wykonywanych fragmentów programu.
 - kilkaset do kilkudziesięciu tysięcy wykonań w zależności od rodzaju optymalizacji.
- Duży koszt czasowy uruchomienia JIT, profilowania i optymalizacji kodu.
- Użycie JIT opłacalne głównie dla aplikacji „serwerowych”:
 - czas pracy aplikacji liczony w godzinach,
 - wielokrotne wykonywanie tych samych fragmentów kodu.

Kompilator Just-in-Time (JIT)

- Techniki optymalizacji kodu:
 - wstawienie ciała metody zamiast jej wywołania (ang. *inlining*),
 - eliminacja wywołań rekurencyjnych,
 - predykcja skoków,
 - wykorzystanie rejestrów procesora,
 - optymalizacja pętli (rozwijanie itp.),
 - eliminacja martwego kodu,
 - **kompilacja do kodu natywnego** (może przyspieszyć kilkadziesiąt razy)
 - ...i kilkanaście innych technik.
- Optymalizacja może zostać w dowolnym momencie cofnięta, jeśli przestaje przynosić korzyść.

Oczyszczanie pamięci (ang. *Garbage Collection*) – metody

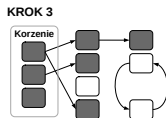
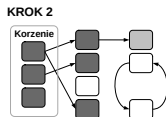
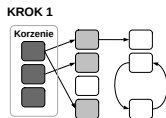
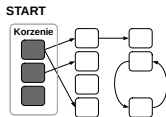
- Oczyszczanie pamięci nie jest ujęte w specyfikacji JVM.
- Dana JVM często dysponuje kilkoma modułami GC.
- Dwa podstawowe podejścia wykorzystywane w różnych językach programowania wykorzystują:
 - zliczanie referencji do obiektu (ang. *reference-counting GC*) – np. Swift, Python,
 - śledzenie osiągalnych obiektów (ang. *tracing GC*) – Java.

Oczyszczanie pamięci oparte na zliczaniu referencji

- Obiekt jest potrzebny („żywy”), dopóki wskazuje na niego co najmniej jedna referencja.
- Liczba referencji do obiektu z reguły przechowywana w samym obiekcie (np. Python).
- Obiekt jest usuwany niezwłocznie, gdy liczba referencji do niego spadnie do zera.
- Zalety:
 - pamięć jest odzyskiwana na bieżąco,
 - **fragmentacja pamięci** stosunkowo niewielka.
- Wady:
 - konieczna *bieżąca* aktualizacja licznika referencji (wydajność!),
 - problem z cyklicznymi referencjami (np. listy dwukierunkowe lub cykliczne).

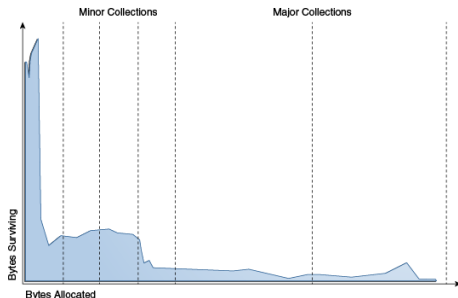
Oczyszczanie pamięci oparte na śledzeniu osiągalnych obiektów

- Obiekt jest potrzebny, jeśli odwołuje się do niego *potrzebny* obiekt.
- Pewne obiekty są na pewno potrzebne („**korzenie**”), m.in.:
 - zmienne statyczne,
 - zmienne lokalne (na stosach wywołań),
 - działające wątki.
- GC przeszukuje graf odwołań, począwszy od korzeni:
 - obiekty osiągalne są oznaczane,
 - na końcu usuwane są nieoznaczone obiekty,
 - stąd inna nazwa – **mark and sweep**.
- GC musi być włączany okresowo lub jeśli brakuje pamięci.
- Wady:
 - okresowe „duże porządki” obciążają system,
 - ryzyko fragmentacji pamięci (w Javie zminimalizowane).



Oczyszczanie pamięci na stercie w JVM – idea

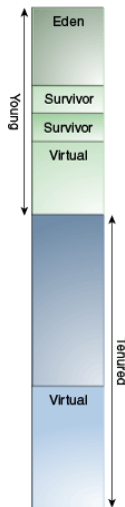
- Wszystkie obecne JVM wykorzystują GC ze śledzeniem.
- Histogram czasu życia obiektów →
- Większość obiektów ma charakter tymczasowy.
- Spostrzeżenie: jeśli obiekt nie został zniszczony bardzo szybko, to zapewne będzie potrzebny przez dłuższy czas.



Źródło: <https://docs.oracle.com/javase/8/docs/technotes/guides/vm/gctuning/generations.html>

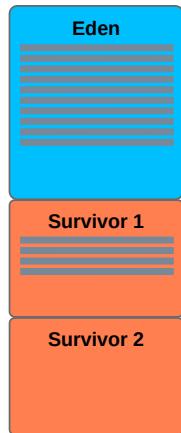
Oczyszczanie pamięci na stercie w JVM – przebieg

- Sterta podzielona na dwie generacje:
 - Młodą (ang. *young*) – częste oczyszczanie.
 - Starszą (ang. *tenured*) – rzadsze oczyszczanie,
 - Obszary oznaczone *virtual* są dostępne (jeszcze niewykorzystane).
- Obiekty są tworzone w obszarze *Eden* części „młodej”.
- Jeden z obszarów *Survivor* jest używany, drugi pozostaje pusty.
- Okresowe oczyszczanie części „młodej”:
 - Znalezienie wszystkich „żywych” obiektów w używanym obszarze *Survivor* i *Eden*.
 - Skopiowanie ich do pustego obszaru *Survivor* (likwiduje fragmentację!).
 - Usunięcie całej zawartości *Eden* i używanego dotychczas *Survivor*.
- Najstarsze obiekty są okresowo przenoszone do części „starszej”.

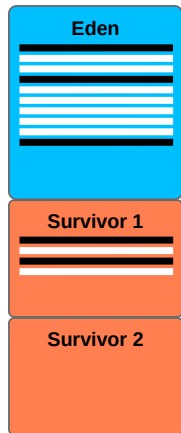


Oczyszczanie pamięci na stercie w JVM – przebieg

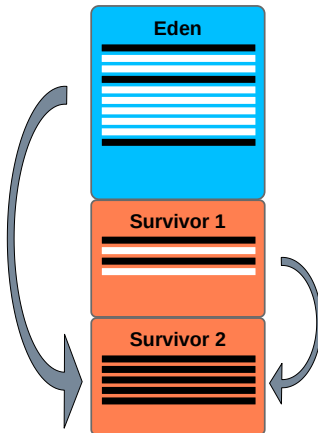
Przed
czyszczeniem



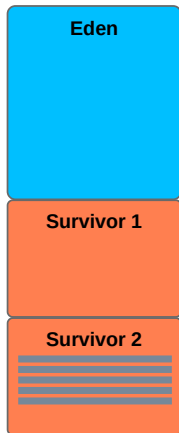
Identyfikacja
„żywych” obiektów



Kopiowanie
„żywych” obiektów



Czyszczenie
Eden i Survivor 1



Oczyszczanie pamięci w JVM – koszt

UWAGA!

- Oczyszczanie pamięci w Javie jest operacją kosztowną.
- Należy unikać częstego tworzenia dużej liczby obiektów tymczasowych.

Spis treści

- 1 Java – maszyna wirtualna
- 2 Java – styl, instrukcje sterujące, operatory

Java – pierwszy program

```
// Java - nasz pierwszy program  
public class Pierwsza {  
    public static void main(String[] args) {  
        System.out.println("Pierwsza klasa!");  
    }  
}
```

- Kod umieszczamy w pliku **Pierwsza.java** (koniecznie!)
- Kompilujemy: **javac Pierwsza.java**
- Uruchamiamy: **java Pierwsza**

Java – nazwy identyfikatorów

- Można używać dowolnych *liter* dostępnych w Unicode oraz cyfr i znaku podkreślenia:
 - `float wyróżnik_Δ_1` – dobrze,
 - `class I♥Java` – niedobrze.
- Nazwy zaczynamy literą.
 - Teoretycznie można zaczynać też `_` lub `$` (zła praktyka).
- W zapisie dużych liczb warto używać podkreśleń: `long milion = 1_000_000;`
- **Klasy** – styl *CamelCase*: `ŁadnaNazwaKlasy`.
- **Zmienne i metody** – styl *mixedCase*: `ładnaNazwaZmiennej`.
- **Stałe** – wersaliki, słowa oddzielone podkreśleniami (*snake_case*): `ŁADNA_NAZWA_STALEJ`.

Java – instrukcja warunkowa `if ... else`

```
if (punkty >= 91) {  
    ocena = 5.0;  
} else if (punkty >= 81) {  
    ocena = 4.5;  
} else if (punkty >= 71) {  
    ocena = 4.0;  
} else if (punkty >= 61) {  
    ocena = 3.5;  
} else {  
    ocena = 3.0;  
}
```

Nawiasy klamrowe są zbędne, jeśli po sprawdzeniu warunku wykonana ma być tylko jedna instrukcja (jak w tym przykładzie).

Złożone instrukcje warunkowe – *short circuiting*

- Złożone wyrażenia logiczne (koniunkcje i alternatywy warunków) są ewaluowane od lewej do prawej strony.
- Ewaluacja jest *przerywana* w momencie, gdy wartość *całego wyrażenia* staje się znana (jak w większości języków programowania).
- Jakie implikacje ma to dla programisty?

Java – instrukcja wyboru `switch ... case`

```
int kolor = 1;

switch (kolor) {
    case 0:
        System.out.println("Czerwony");
        break;
    case 1:
        System.out.println("Zielony");
        break;
    case 2:
        System.out.println("Niebieski");
        break;
    default:
        System.out.println("Inny");
}
```

Java – instrukcja wyboru `switch ... case`

```
int kolor = 1;

switch (kolor) {
    case 0:
        System.out.println("Czerwony");
        break;
    case 1:
        System.out.println("Zielony");
        break;
    case 2:
        System.out.println("Niebieski");
        break;
    default:
        System.out.println("Inny");
}
```

```
int kolor = 1;

if (kolor == 0) {
    System.out.println("Czerwony");
}
else if (kolor == 1) {
    System.out.println("Zielony");
}
else if (kolor == 2) {
    System.out.println("Niebieski");
}
else {
    System.out.println("Inny");
}
```

Java – operator trójargumentowy

Konstrukcję:

```
if (warunek == true)
    odpowiedź = "PRAWDA";
else
    odpowiedź = "NIEPRAWDA";
```

można zastąpić **operatorem trójargumentowym** znanym z języka **C**:

```
String odpowiedź = (warunek == true) ? "PRAWDA" : "NIEPRAWDA";
```

Java – pętla **for**

Jak iterować po elementach tablicy?

```
char[] tab = { 'J', 'a', 'v', 'a' };
```

- Przez indeksowanie – składnia identyczna jak w języku **C**:

```
for (int i = 0; i < 4; i++) { System.out.println(tab[i]); }
```

- Sposób zalecany *wyłącznie* gdy indeks jest do czegoś potrzebny.

- Przez bezpośrednie odwołanie do elementów – jak w języku **C++** lub **Python**:

```
for (char znak : tab) { System.out.println(znak); }
```

- Sposób użyteczny również przy iterowaniu po **kolekcjach** (listach, zbiorach itp.)

Java – zagnieżdżona pętla **for** – formalnie

```
for (int i = 0; i < 4; i++) {  
    for (int j = 0; j < 3; j++) {  
        for (int k = 0; k < 2; k++) {  
            System.out.println("To się wyświetli 24 razy");  
        }  
    }  
}  
  
System.out.println("A to tylko raz");
```

Java – zagnieżdżona pętla **for** – inaczej

```
for (int i = 0; i < 4; i++)  
    for (int j = 0; j < 3; j++)  
        for (int k = 0; k < 2; k++)  
            System.out.println("To się wyświetli 24 razy");  
System.out.println("A to tylko raz");
```


Java – zagnieżdżona pętla **for** – brzydko

```
for (int i = 0; i < 4; i++)  
    for (int j = 0; j < 3; j++)  
        for (int k = 0; k < 2; k++)  
            System.out.println("To się wyświetli 24 razy");  
            System.out.println("A to tylko raz");
```

Dygresja – przeciążanie operatorów

- W wielu językach (ale nie Javie!) programista może **przeciążać** wybrane operatory (arytmetyczne, logiczne itp.), tj. definiować ich działanie dla różnych „niestandardowych” typów argumentu.
- Użycie operatorów jest często intuicyjne: `"ab" + "cd" + "ef" → "abcdef"`.
 - Bardziej czytelne i łatwiejsze do zapisania niż `"ab".append("cd").append("ef")`
- Przeciążone operatory zachowują domyślną *łączność* i *priorytet*:
`A-B+C*D ↔ (A-B)+(C*D)` gdzie `A`, `B`, `C`, `D` – obiekty *dowolnego* typu, dla którego zdefiniowano operatory arytmetyczne.

Przeciążanie operatorów w różnych językach

- **C++**: można definiować znaczenie operatorów dla własnych klas, definiując metody: `operator+()`, `operator&&()`, `operator<()` itp.
- **Python**: należy zdefiniować odpowiednią metodę specjalną, np. `__add__`, `__and__`, `__lt__` itp.
- **Java**: przeciążanie operatorów niemożliwe (na razie).

Przeciążanie operatorów – pułapki

- Znaczenie operatorów dla typów „złożonych” bywa niejednoznaczne.
- **Przykład 1** – „dodawanie” dwóch list: $[1, 2, 3] + [4, 5, 6] \longrightarrow ?$
 - może $[1, 2, 3, 4, 5, 6]$?
 - a może $[5, 7, 9]$?
- Definiujący operatory odpowiada za (nie)respektowanie praw logiki i arytmetyki.
- **Przykład 2:**
 - wg reguł arytmetyki: $x \leq y \iff (x < y) \vee (x = y)$,
 - w programowaniu: wszystko zależy od definicji $\leq$, $<$ i $=$ dla typu, do którego należą x i y .
- **Przykład 3:**
 - prawo wyłączonego środka: $x \vee \neg x$,
 - zasada sprzeczności: $\neg(x \wedge \neg x)$,
 - w programowaniu: wszystko zależy od definicji operatorów negacji, alternatywy i koniunkcji dla typu, do którego należy x .